Etapa 11: un programa candidato completo

Pregunta e hipótesis

Ya elegimos una variante a partir de bancos experimentales. ¿Podemos reunirla en un programa claro, correcto y medible sin cambiar el algoritmo? La hipótesis es que la composición conserva la corrección; su rendimiento debe medirse nuevamente.

Qué conservamos

Cholesky right-looking escalar en C; MPI; propiedad 1D cíclica de columnas (b=1); vector local contiguo; generación distribuida y factor completo reunido en rango 0. No añadimos OpenMP, LAPACK ni un algoritmo por bloques.

$$\text{owner}(j) = j \bmod P, \qquad A = LL^T$$

Un programa, dos compilaciones comparables

src/main_candidate.c es la entrada. Con USE_MPI se enlaza con cholesky_mpi.c y cholesky.c. Sin USE_MPI produce una referencia secuencial autónoma con right-looking buffered. Así comparten generador, validación y definición de salida, pero el secuencial no inicia MPI ni deja procesos inactivos esperando.

Entrada: N entre 1 y 20000; semilla fija 2026. Ese límite protege los conteos MPI, no garantiza memoria disponible. No hay lectura de matrices externas. El límite local probado con siete repeticiones es N=2048; N=3072 pertenece al piloto histórico.

Cómo ejecutarlo desde desde_cero, en Linux/WSL

```
make candidate
./build/cholesky_candidate_seq 1024
mpirun --bind-to core --map-by core -np 3
./build/cholesky_candidate_mpi 1024
```

Las dos últimas líneas representan un solo comando. La salida CSV informa tiempos, residuos y validez. L queda completo en memoria del rango 0; no se escribe un archivo de matriz ni se incluye I/O de disco del factor. El triángulo inferior representa L; el superior no forma parte del factor.

Próxima pregunta

Si la integración es correcta, ¿cuántos procesos reducen el tiempo hasta obtener L y cuánto se pierde al medir el comando completo?

Comprobaciones y contrato del reloj

Comprobación	Resultado
Programa: N=1,2,7,31,128; secuencial y P=1,2,3,4,6	30 configuraciones válidas; incluye N menor que P y repartos desiguales.
Residuos	Ocho proyecciones reproducibles; residuo completo para N menor o igual a 128. Límite 1e-12 y control de valores finitos y diagonal positiva.
Biblioteca numérica propia	Pruebas existentes: factor conocido, identidad, variantes, entradas inválidas y detección de corrupción.
Interfaz	Argumentos inválidos rechazados. Errores de reserva abortan el trabajo MPI.

Los tamaños grandes usan proyecciones $O(n^2)$ por vector, no reconstrucción completa $O(n^3)$. Eso es evidencia numérica, no un certificado para cualquier entrada. Esta etapa no amplía la familia de matrices: conserva las SPD diagonalmente dominantes del proyecto.

$$T_{app} = T_{reserva} + T_{generacion} + T_{factor} + T_{reunion}$$

T_app es el intervalo medido en rango 0, desde la reserva hasta que reúne el resultado. También se registra la máxima duración local. Las fases de rango 0 suman su total, pero no son tiempos puros: por ejemplo, factor incluye comunicaciones y esperas. No sumamos máximos de fases de rangos distintos.

La comprobación regenera A en rango 0 y calcula residuos después de T_app. Es la misma operación matemática en ambas compilaciones. Se mide aparte y queda incluida en el reloj externo, junto con liberación, salida CSV y cierre. No se genera una referencia secuencial adicional en el MPI candidato.

Decisión y alcance

Consolidamos este ejecutable como candidato operativo; los compare_*.c quedan como evidencia histórica. El programa no elige P automáticamente. Tampoco sustituye el estudio de rendimiento en el clúster.

Trazabilidad: resultados/cierre_local_20260924/tests, build_tests.log y fuentes congeladas. Generación y medición en C; ejecución en Bash; análisis y documentación en Python. Desarrollo asistido por IA, con pruebas ejecutadas.